

Sauce Demo Defect Catalog

Documented bugs observed on <https://www.saucedemo.com/> , captured by exercising every logging-in user across login → inventory → sort → cart → checkout → finish.

How findings were collected

A Playwright diagnostic logged in as `locked_out_user` (login only) and as each of the five users that can log in, and for each user recorded:

- inventory item count, names, prices, descriptions, image `src` , button initial state
- displayed order and displayed prices after each of the four sort options (`az` , `za` , `lohi` , `hilo`)
- product detail page (image, description, price, URL) for every one of the six items
- before/after button text and cart-badge value for **every one of the six products** on Add to cart, then on Remove from the inventory page
- cart contents (items, descriptions, prices, quantities, button states) after adding all six products
- continue-shopping and cancel-button destinations
- checkout-information form behaviour: empty submit, only-firstName submit, only-firstName+lastName submit, all-fields submit, lastName cross-binding probe (typing into firstName while lastName is empty), input read-back of every field
- checkout-overview totals (item total + tax = total) and finish-button outcome (URL, page errors, console errors, complete-header text, back-home navigation, post-order cart badge)
- burger-menu link `href` values, footer link `href` values, copyright text
- Reset App State on a fully-loaded inventory: cart-badge before/after and per-item button text before/after for all six products
- logout: post-logout URL and whether the session is enforced (visiting `/inventory.html` directly after logout)

Raw observations are written to `test-results/sauce-demo-diagnostic/<user>.json` (gitignored) when the diagnostic is run. `standard_user` is treated as the working baseline. Anything diverging from it on a non-performance user is considered a defect unless explicitly part of that user's design.

Summary

[illegible]

error_user	OK	OK	OK	Broken	Silent fail on 3/6	Broken	OK	Skips last-name validation	Finish throws JS error	Mu def
visual_user	OK	First item placeholder	Randomised on inventory only	Order correct, displayed prices random	OK	OK	OK	OK	OK	Inv disj def & c pric cor

Plus one cross-user defect: **Reset App State** does not clear per-item button state on the inventory page (verified on every user that can log in).

Cross-user defects

CU-1: Reset App State leaves the per-item Add to cart / Remove button state intact

- **Affects:** confirmed independently on every user that can log in (`standard_user` , `problem_user` , `performance_glitch_user` , `error_user` , `visual_user`).
- **Steps to reproduce:**
 1. Log in as any non-locked-out user.
 2. Add any item(s) to the cart from the inventory page.
 3. Open the burger menu and click **Reset App State**.
- **Observed:** the cart badge disappears, but every item that was previously added still shows the **Remove** button. Diagnostic captured `buttonsBefore` and `buttonsAfter` on all five users — they are byte-for-byte identical after Reset.
- **Expected:** the inventory page should also reset every item's button text back to **Add to cart**.
- **Severity:** functional; misleads users into thinking items are still in their cart even though the cart is empty.
- **Coverage:** asserted by `tests/targets/sauce-demo/ui/sauce-demo-ui.spec.ts` → Known defects → `reset app state does not clear per-item Add to cart / Remove button state for any product` (currently only covers `standard_user` ; could be parameterised to cover all four affected users).

problem_user

PU-1: Inventory images all render the 404 placeholder

- **Steps to reproduce:**
 1. Log in as `problem_user` .
 2. Inspect the inventory item images.
- **Observed:** every product card shows `/static/media/s1-404.168b1cce10384b857a6f.jpg` (a placeholder dog face). Six items, one unique image.
- **Expected:** each of the six products should show its dedicated image (six unique srcs as seen for `standard_user`).
- **Severity:** visual / data integrity. Users cannot visually distinguish products on the catalogue page.

PU-2: Sort dropdown does not reorder the inventory

- **Steps to reproduce:**
 1. Log in as `problem_user` .

2. Note the displayed order of items.
 3. Select each of the four sort options in turn.
- **Observed:** the dropdown's selected option visually changes, but the displayed inventory remains in the default order for all four sort options. Both names and prices stay in the identical default sequence.
 - **Expected:** Z-A should reverse, lohi should ascend by price, hilo should descend by price.
 - **Severity:** functional. The sort feature is non-functional for this user.
 - **Coverage:** asserted by `tests/targets/sauce-demo/ui/sauce-demo-ui.spec.ts` → Known defects → `problem_user: inventory sort dropdown does not actually reorder items`.

PU-3: Checkout last-name input writes into the first-name field

- **Steps to reproduce:**
 1. Log in as `problem_user`, add any item, open cart, click Checkout.
 2. Fill First Name with `Diag`. Read the value back — it is `Diag`. ✓
 3. Fill Last Name with `Nostic`.
 4. Read both inputs back via `inputValue()`.
- **Observed:**
 - First Name now reads `Nostic` (the value just typed into Last Name).
 - Last Name reads empty string.
 - Continue button surfaces `Error: Last Name is required` and the URL stays on `/checkout-step-one.html`.
 - The cross-binding is one-directional: typing into First Name does **not** mutate Last Name (verified by clearing Last Name, typing into First Name, and re-reading Last Name — it stays empty).
- **Expected:** Last Name should accept and retain its own value; the form should advance to `/checkout-step-two.html` once all three fields are filled.
- **Severity:** blocker. `problem_user` cannot complete a checkout regardless of input.
- **Possible root cause:** the Last Name input's `onChange` handler is wired to the First Name field's state setter (or both inputs share the same `name` / `id` and the React binding picks the first match). Sharper than the original "lastName rejects input" framing — the typed text is **not** discarded, it just lands in the wrong field.

PU-4: Add to cart silently fails on three of the six products

- **Steps to reproduce:**
 1. Log in as `problem_user`.
 2. Click **Add to cart** on each of the six inventory items in turn.
 3. After each click, read the button text and the cart badge.
- **Observed:**
 - Backpack, Bike Light, Onesie → button flips to **Remove**, badge increments. ✓
 - Bolt T-Shirt, Fleece Jacket, Test.allTheThings() T-Shirt (Red) → button stays as **Add to cart**, badge does **not** increment, item is **not** added to the cart.
 - Confirmed by visiting `/cart.html` after attempting all six: cart contains exactly 3 items (Backpack, Bike Light, Onesie).
- **Expected:** every Add to cart click should add the item and increment the badge.
- **Severity:** blocker. Half of the catalogue cannot be purchased.

PU-5: Remove button on the inventory page is non-functional

- **Steps to reproduce:**
 1. Log in as `problem_user`. Add Backpack, Bike Light, and Onesie successfully (badge = 3).
 2. Click **Remove** on Backpack on the inventory page.
 3. Read the button text and the cart badge.

- **Observed:** button text stays as **Remove**, cart badge stays at 3. The same is true for Bike Light and Onesie. None of the items can be removed from the inventory page.
- **Expected:** Remove should flip the button back to **Add to cart** and decrement the badge.
- **Severity:** functional. The user cannot back out of the cart from the inventory page; they would have to go through `/cart.html` to remove items, which the diagnostic did not exhaustively re-verify.

PU-6: Inventory item title links are off-by-one

- **Steps to reproduce:**
 1. Log in as `problem_user`.
 2. For each inventory item, click the title and observe the resulting URL and rendered detail.
- **Observed:** every link navigates to the next item's id (id+1), and the highest-id item rolls into a non-existent id and renders the placeholder image.

Clicked item	Standard id	problem_user id clicked	Detail page shown
Sauce Labs Bike Light	0	1	Bolt T-Shirt detail
Sauce Labs Bolt T-Shirt	1	2	Onesie detail
Sauce Labs Onesie	2	3	Test.allTheThings() detail
Test.allTheThings() T-Shirt (Red)	3	4	Backpack detail
Sauce Labs Backpack	4	5	Fleece Jacket detail
Sauce Labs Fleece Jacket	5	6	404 placeholder

- **Expected:** clicking each title should open that item's own detail page.
- **Severity:** functional / data integrity. Combined with PU-1 (every inventory image is the placeholder) the user has no reliable way to identify which product is which on the catalogue page.

performance_glitch_user

PG-1: All flows succeed but are dramatically slow

- **Steps to reproduce:** log in as `performance_glitch_user` and exercise any flow.
- **Observed:** the same diagnostic took 31.4 s for `performance_glitch_user` versus 1.4 s for `standard_user` — roughly 22× slower. Every page transition (login, inventory render, cart navigation, checkout) shows multi-second delays.
- **Expected:** comparable timings to `standard_user`.
- **Severity:** non-functional / performance.
- **Notes:** functional output is identical to `standard_user` (correct images, descriptions, sort, prices, totals, completion message). The defect is delay only.

error_user

EU-1: Sort dropdown does not reorder the inventory

- Same shape as `problem_user` PU-2: all four sort options (`az` , `za` , `lohi` , `hilo`) leave the inventory in the identical default order; both name and price sequences match the unsorted state.
- **Severity:** functional.
- **Note:** the existing `Known defects` regression test only logs in as `problem_user` , so this user is not yet covered by an automated assertion.

EU-2: Checkout last-name validation incorrectly accepts an empty value

- **Steps to reproduce:**
 1. Log in as `error_user`, add any item, open cart, click Checkout.
 2. Fill First Name and Postal Code; attempt to fill Last Name.
 3. Click Continue.
- **Observed:**
 - Just like `problem_user`, the `lastName` field reads back as empty after filling.
 - Unlike `problem_user`, clicking Continue advances to `/checkout-step-two.html` with no validation error.
- **Expected:** the form should reject an empty last name and stay on step one with an error message (matching `standard_user` behaviour).
- **Severity:** functional / data integrity. Orders can be created with no last name attached.

EU-3: Finish button throws a JavaScript error and never completes the order

- **Steps to reproduce:**
 1. Log in as `error_user`, add an item, proceed through the checkout-info page (`lastName` ends up empty, see EU-2), arrive on the checkout-overview page.
 2. Click Finish.
- **Observed:**
 - Page error fires: `Ye.cesetRart is not a function` (a deliberate-looking misspelling of `resetCart`).
 - URL stays on `/checkout-step-two.html`; the `complete-header` element never appears.
 - Subsequent console errors:
 - Access to fetch at 'https://submit.backtrace.io/UNIVERSE/TOKEN/json' from origin 'https://www.saucedemo.com' has been blocked by CORS policy (×2).
 - Failed to load resource: net::ERR_FAILED (×2).
 - Failed to load resource: the server responded with a status of 401 (Unauthorized) (×2).
- **Expected:** Finish should clear the cart and navigate to `/checkout-complete.html` showing **Thank you for your order!**.
- **Severity:** blocker. `error_user` cannot complete an order.
- **Note:** the CORS / 401 errors are the error-reporting backend (Backtrace) being unreachable; they are a side effect, not the root cause. The page-error message is the actionable signal.

EU-4: Add to cart silently fails on three of the six products

- Same product set and same symptoms as PU-4: clicking Add to cart on Bolt T-Shirt, Fleece Jacket, or Test.allTheThings() T-Shirt (Red) leaves the button as **Add to cart**, leaves the cart badge unchanged, and does not add the item to the cart.
- Confirmed by inspecting `/cart.html` after attempting all six: cart contains exactly 3 items (Backpack, Bike Light, Onesie).
- **Severity:** blocker. Same impact as PU-4.

EU-5: Remove button on the inventory page is non-functional

- Same shape as PU-5: clicking **Remove** on Backpack, Bike Light, or Onesie (the items that were successfully added) does not change the button text and does not decrement the cart badge.
- **Severity:** functional.

EU-6: Checkout-information form skips Last Name validation entirely

- **Steps to reproduce:**
 1. Log in as `error_user`, add any item, open cart, click Checkout.
 2. Fill First Name only and click Continue.

- **Observed:** the validation error is `Error: Postal Code is required` — the form jumps straight from First Name to Postal Code, skipping Last Name validation altogether.
 - When all three fields are filled, Last Name still reads empty (same input-not-persisting symptom as `problem_user`'s PU-3, but without the cross-binding into First Name), the form advances to `/checkout-step-two.html` anyway, and the order is created with no last name attached. (Already documented in EU-2; EU-6 is the *validation-message* angle of the same defect surface.)
 - **Expected:** Continue with only First Name filled should fail with `Error: Last Name is required` (matching `standard_user` behaviour).
 - **Severity:** functional / data integrity.
-

visual_user

VU-1: First inventory item image is the 404 placeholder

- **Steps to reproduce:** log in as `visual_user` and inspect the inventory item images.
- **Observed:** the first item (Sauce Labs Backpack) renders `/static/media/sl-404.168b1cce10384b857a6f.jpg`. The other five items render their correct images.
- **Expected:** Backpack should render `/static/media/sauce-backpack-1200x1500.<hash>.jpg`.
- **Severity:** visual.

VU-2: Inventory page prices are randomised on every render

- **Steps to reproduce:**
 1. Log in as `visual_user`.
 2. Read the displayed prices on the inventory page.
 3. Apply each sort option in turn and re-read the displayed prices.
- **Observed:** the displayed prices change on every render and never match the real product prices. From one captured run:
 - Initial: `[97.76, 63.95, 44.69, 22.25, 39.04, 60.08]`
 - After A→Z sort: `[41.15, 39.59, 59.96, 16.99, 12.35, 97.08]`
 - After Z→A sort: `[92.63, 47.6, 78.19, 35.48, 92.63, 97.36]`
 - After lohi sort: `[97.37, 81.63, 88.73, 21.76, 79.12, 67.97]`
 - After hilo sort: `[60.76, 20.59, 45.76, 90.37, 19.51, 28.2]`
- **Expected:** stable real prices `[29.99, 9.99, 15.99, 49.99, 7.99, 15.99]` for `standard_user`.
- **Severity:** data integrity. Price displayed on the catalogue is meaningless.

VU-3: Sort uses real prices but displays randomised prices

- **Steps to reproduce:** as above; specifically observe lohi/hilo.
- **Observed:** when sorting low-to-high or high-to-low, the **item order** matches what `standard_user` produces (i.e. the catalogue sort uses the real prices). However, the **displayed prices** alongside each card are still random and do not reflect any low-to-high or high-to-low ordering.
- **Expected:** displayed prices and sort order should agree.
- **Severity:** data integrity / consistency.

VU-4: Cart and checkout prices are correct despite VU-2 / VU-3

- **Observed:** in the cart and on the checkout overview, prices revert to the real values (`$29.99` , `$9.99` , `$7.99`), and the subtotal / total compute correctly to `$47.97` / `$51.81`.
- **Note:** this confirms VU-2/VU-3 are inventory-page display defects only and do not corrupt the underlying product data.

Investigation gaps

The second exhaustive pass closed most of the original gaps. What still remains:

- **visual_user CSS / layout / colour defects** — not detectable via DOM-level inspection. Would need screenshot comparison against `standard_user` (full-page screenshots for `visual_user` are saved at `test-results/sauce-demo-diagnostic/visual_user/*.png` when the diagnostic is re-run, but a baseline diff is not yet performed).
- **Cart-page Remove button on `problem_user` / `error_user`** — PU-5 and EU-5 confirm the inventory-page Remove is broken; the cart-page Remove was not exhaustively re-verified per item.
- **Inventory item description fidelity for every product per user** — the diagnostic captures full descriptions in `cartAfterAddingAll` and they match `standard_user`, but a row-by-row diff per inventory item per user has not been formally produced.
- **Order ID and persistence after Finish** — the diagnostic checks `complete-header` text and `Back Home` navigation, but does not verify any server-side order record (Sauce Demo has no real backend, so this is largely vacuous).
- **Mobile / responsive viewport** — diagnostic ran with desktop viewport only.

Closed since the previous catalogue: product detail page rendering per user (PU-6 surfaced), `problem_user` first/last name cross-binding (PU-3 sharpened), Add to cart for every product (PU-4 / EU-4 surfaced), Reset App State on every user (CU-1 confirmed cross-user), logout behaviour (verified working and session enforced for every user).